

The JCommon Class Library

Written by David Gilbert

January 25, 2002

© 2001, 2002, by Simba Management Limited. All rights reserved.

*Everyone is permitted to copy and distribute verbatim copies of this document,
but changing it is not allowed.*

Contents

1	Introduction	5
1.1	What is JCommon?	5
1.2	Licence	5
1.3	Latest Version	5
1.4	Automated Testing	5
1.5	Acknowledgements	6
1.6	Comments and Suggestions	6
2	Package Overview	7
3	Package: com.jrefinery.data	8
3.1	AbstractDataset	8
3.2	AbstractSeriesDataset	8
3.3	BasicTimeSeries	9
3.4	CategoryDataset	10
3.5	CombinationDataset	10
3.6	CombinedDataset	11
3.7	Dataset	11
3.8	DatasetChangeEvent	11
3.9	DatasetChangeListener	12
3.10	Datasets	12
3.11	Day	13
3.12	DefaultCategoryDataset	14
3.13	DefaultPieDataset	14
3.14	DefaultXYDataset	15
3.15	DomainInfo	15
3.16	HighLowDataset	15
3.17	Hour	16
3.18	IntervalXYDataset	17
3.19	IntervalXYZDataset	17
3.20	Millisecond	17
3.21	Minute	18
3.22	Month	19
3.23	PieDataset	20
3.24	Quarter	21
3.25	RangeInfo	22
3.26	Second	23
3.27	SeriesChangeListener	24
3.28	SeriesDataset	24
3.29	SeriesException	24
3.30	Statistics	24
3.31	SubseriesDataset	25
3.32	TimePeriod	25
3.33	TimePeriodFormatException	26
3.34	TimeSeriesCollection	26
3.35	TimeSeriesDataPair	27
3.36	TimeSeriesTableModel	27
3.37	Values	28

3.38	Week	28
3.39	XYDatapair	29
3.40	XYDataset	29
3.41	XYSeries	30
3.42	XYSeriesCollection	30
3.43	XYZDataset	30
3.44	Year	30
4	Package: com.jrefinery.date	32
4.1	AnnualDateRule	32
4.2	DayAndMonthRule	32
4.3	DayOfWeekInMonthRule	32
4.4	EasterSundayRule	33
4.5	RelativeDayOfWeekRule	33
4.6	SerialDate	33
4.7	SerialDates	33
4.8	SpreadsheetDate	34
5	Package: com.jrefinery.io	35
5.1	Files	35
6	Package: com.jrefinery.layout	36
6.1	CenterLayout	36
6.2	FormatLayout	36
6.3	LCBLayout	36
7	Package: com.jrefinery.ui	37
7.1	AboutFrame	37
7.2	AboutPanel	39
7.3	BevelArrowIcon	39
7.4	Contributor	39
7.5	ContributorsPanel	39
7.6	ContributorsTableModel	39
7.7	DateChooserPanel	40
7.8	ExtensionFileFilter	40
7.9	FontChooserDialog	40
7.10	FontChooserPanel	40
7.11	FontDisplayField	40
7.12	InsetsChooserPanel	40
7.13	InsetsTextField	40
7.14	IntegerDocument	40
7.15	JRefineryUtilities	40
7.16	L1R1ButtonPanel	41
7.17	L1R2ButtonPanel	41
7.18	L1R3ButtonPanel	41
7.19	Licences	41
7.20	NumberCellRenderer	42
7.21	PaintSample	42
7.22	SerialDateChooserPanel	42
7.23	Size2D	42

7.24	SortableTable	42
7.25	SortableTableHeaderListener	42
7.26	SortableTableModel	42
7.27	SortButtonRenderer	43
7.28	StandardDialog	43
7.29	StrokeChooserPanel	43
7.30	StrokeSample	43
7.31	SystemPropertiesFrame	44
7.32	SystemPropertiesPanel	45
7.33	SystemPropertiesTableModel	45
7.34	WizardDialog	45
7.35	WizardPanel	46
8	Package: com.keypoint	47
8.1	PNGEncoder	47
A	The GNU Lesser General Public Licence	48

1 Introduction

This document provides information about the JCommon Class Library, version 0.5.3.

1.1 What is JCommon?

JCommon is a free class library containing miscellaneous utility classes written in Java. You can download the library from:

<http://www.jrefinery.com/jcommon>

JCommon should compile and run on any implementation of the Java 2 platform. You can use JCommon in your own projects, subject to the terms of the GNU Lesser General Public Licence (see the next section).

1.2 Licence

JCommon is licensed using the GNU Lesser General Public Licence (LGPL). Appendix A contains the full text of the licence.

You can find out more information about the LGPL, free software, and open source software, at the following web-sites:

Organisation:	Web Address:
The Free Software Foundation	http://www.fsf.org
The Open Source Initiative	http://www.opensource.org

1.3 Latest Version

This document has been written for version 0.5.3 of JCommon. The latest version, plus any bug reports or other announcements, can be found at:

<http://www.jrefinery.com/jcommon>

Some other open source projects are also hosted on this web site.

1.4 Automated Testing

The JCommon source code now includes some automated tests utilising the JUnit testing framework. You can run these tests by invoking the JUnit test runner on the following class:

`com.jrefinery.JCommonTestSuite`

For more information about JUnit, refer to:

<http://www.junit.org>

The tests are not comprehensive, but additional tests will be added in the future.

1.5 Acknowledgements

Thanks to the following people who have contributed to JCommon:

- Nabuo Tamemasa—the classes used to implement sortable tables (`Bevel-
ArrowIcon`, `SortButtonRenderer`, `SortableTableHeaderListener` and `SortableTableModel`) are based on code by Nabuo Tamemasa, posted at <http://www.codeguru.com>;
- Andrzej Porebski—contributed several classes for editing an `Insets` object;
- J. David Eisenberg—the `PNGEncoder` class;
- Mark Watson—several classes in the `com.jrefinery.data` package;
- Matthew Wright—the `Statistics` class, originally submitted to the JFreeChart project;

1.6 Comments and Suggestions

If you have any comments or suggestions regarding the JCommon Class Library, or this document, please post a message to the JCommon discussion forum (preferred) or send an e-mail to: david.gilbert@jrefinery.com.

2 Package Overview

This section contains special notes and reference information for all the classes in the JCommon class library.

Package:	Description:
<code>com.jrefinery.data</code>	Data related classes.
<code>com.jrefinery.date</code>	Date related classes.
<code>com.jrefinery.io</code>	I/O related classes.
<code>com.jrefinery.layout</code>	Custom layout managers.
<code>com.jrefinery.ui</code>	Useful user interface classes.
<code>com.keypoint</code>	PNG encoder.

Additional information can be found in the Javadoc-generated HTML documentation for JCommon.

3 Package: com.jrefinery.data

This package contains data classes, including time series classes, and the standard interfaces used by JFreeChart.

3.1 AbstractDataset

3.1.1 Overview

A useful base class for implementing the **Dataset** interface (or extensions). This class provides a default implementation of the *change listener* mechanism.

3.1.2 Constructors

The default constructor:

```
protected AbstractDataset();  
Allocates storage for the registered change listeners.
```

3.1.3 Methods

```
public void addChangeListener(DatasetChangeListener listener);  
Registers a change listener with the dataset. The listener will be notified  
whenever the dataset changes.  
  
public void removeChangeListener(DatasetChangeListener listener);  
Deregisters a change listener. The listener will no longer be notified  
whenever the dataset changes.
```

3.1.4 Notes

You can implement a dataset without subclassing **AbstractDataset**. This class is provided simply for convenience to save you having to implement your own change listener mechanism.

See Also

Dataset, **DatasetChangeListener**, **AbstractSeriesDataset**.

3.2 AbstractSeriesDataset

3.2.1 Overview

A useful base class for implementing the **SeriesDataset** interface (or extensions). This class extends **AbstractDataset**.

3.2.2 Constructors

The default constructor:

```
protected AbstractSeriesDataset();  
Simply calls the constructor of the superclass.
```

3.2.3 Methods

Implementations are provided for the following methods:

```
public String[] getLegendItemLabels();  
Returns an array of series names.
```

3.2.4 Notes

You can implement a dataset without subclassing `AbstractSeriesDataset`. This class is provided simply for convenience.

See Also

`Dataset`.

3.3 BasicTimeSeries

3.3.1 Overview

A time series is a data structure that associates numeric values with particular time periods. In other words, a collection of data values in the form *(timeperiod, value)*.

The time periods are represented by subclasses of `TimePeriod`. Subclasses include `Year`, `Quarter`, `Month`, `Week`, `Day`, `Hour`, `Minute`, `Second` and `Millisecond`. Different subclasses of `TimePeriod` cannot be mixed in one time series.

A time series may contain zero, one or many time periods with associated data values.

3.3.2 Constructors

There are three constructors:

```
public BasicTimeSeries(String name);  
Creates an empty time series for daily data (that is, one value per day).  
public BasicTimeSeries(String name, Class timePeriodClass);  
Creates an empty time series. The caller specifies the time period.  
public BasicTimeSeries(String name, String domain, String range, Class  
timePeriodClass);  
Creates an empty time series. The caller specifies the time period, plus  
strings describing the domain and range.
```

3.3.3 Methods

```
public void add(TimePeriod period, Number value) throws SeriesException;  
Adds a new value to the time series. Throws an exception if the time pe-  
riod is not unique within the series.
```

3.3.4 Notes

The class name was formerly `TimeSeries`, but this has been changed to avoid confusion with the subclass in the `com.jrefinery.finance` package.

See Also

`TimeSeriesCollection`

3.4 CategoryDataset

3.4.1 Overview

An interface (extending `SeriesDataset`) that defines the structure of a *category dataset*. The dataset consists of a table of series and categories. A value is associated with each combination of series and category (`null` values are permitted).

3.4.2 Methods

To obtain the number of categories:

```
public int getCategoryCount();
```

Returns the number of categories in the dataset.

To get a list of the categories in the dataset:

```
public List getCategoryNames();
```

Returns a list of the categories in the dataset.

To get the value for a series/category combination:

```
public Number getValue(int series, Object category);
```

Returns the value associated with a particular series and category. The value may be `null`.

3.4.3 Notes

You can use any `Object` instance to represent a category. Using `String` is convenient, as the `toString()` method is used whenever a label is required for a category.

This interface is intended for reading data, not updating it.

See Also

`DefaultCategoryDataset`, `SeriesDataset`.

3.5 CombinationDataset

3.5.1 Overview

An interface for combining datasets. Written by Bill Kelemen.

3.5.2 Notes

This interface is used to create combined charts with the `JFreeChart` class library.

See Also

`CombinedDataset`.

3.6 CombinedDataset

3.6.1 Overview

An implementation of the `CombinationDataset` interface. Written by Bill Kelemen.

3.6.2 Notes

This class is used to create combined charts with the `JFreeChart` class library.

See Also

`CombinationDataset`.

3.7 Dataset

3.7.1 Overview

The base interface for datasets. Not useful in its own right, this interface is further extended by `PieDataset`, `CategoryDataset` and `SeriesDataset`.

3.7.2 Methods

A couple of methods relate to the use of datasets for drawing charts (see `JFreeChart`):

```
public int getLegendItemCount();  
Returns the number of items to display in the legend.  
  
public String[] getLegendItemLabels();  
Returns an array of strings to use as labels in the legend.
```

Two further methods are used for registering change listeners with the dataset:

```
public void addChangeListener(DatasetChangeListener listener);  
Registers a change listener with the dataset.  
  
public void removeChangeListener(DatasetChangeListener listener);  
Deregisters a change listener.
```

3.7.3 Notes

This interface is not intended to be used directly, you should use an extension of this interface such as `PieDataset`, `CategoryDataset` or `XYDataset`.

See Also

`PieDataset`, `SeriesDataset`.

3.8 DatasetChangeEvent

3.8.1 Overview

An event that is used to provide information about changes to datasets.

See Also

`DatasetChangeListener`.

3.9 DatasetChangeListener

3.9.1 Overview

An interface through which dataset change event notifications are posted. If a class needs to receive notification of changes to a dataset, then it needs to implement this interface and register itself with the dataset.

3.9.2 Methods

The interface defines a single method:

```
public void datasetChanged(DatasetChangeEvent event);  
Receives notification of a change to a dataset.
```

See Also

`DatasetChangeEvent`.

3.10 Datasets

3.10.1 Overview

A collection of utility methods for working with datasets.

3.10.2 Methods

To get the minimum and maximum domain values in a dataset:

```
public static Number getMinimumDomainValue(Dataset data);  
Returns the minimum domain value for the dataset.  
  
public static Number getMaximumDomainValue(Dataset data);  
Returns the maximum domain value for the dataset.
```

To get the minimum and maximum range values in a dataset:

```
public static Number getMinimumRangeValue(Dataset data);  
Returns the minimum range value for the dataset.  
  
public static Number getMaximumRangeValue(Dataset data);  
Returns the maximum range value for the dataset.
```

To create a `PieDataset` from a `CategoryDataset`:

```
public static PieDataset createPieDataset(CategoryDataset data, Object  
category);  
Returns a pie dataset by taking all the values in the category dataset for  
the specified category.  
  
public static PieDataset createPieDataset(CategoryDataset data,  
int series) ;  
Returns a pie dataset by taking all the values in the category dataset for  
the specified series.
```

See Also

`DomainInfo`, `RangeInfo`.

3.11 Day

3.11.1 Overview

A subclass of `TimePeriod` that represents one day. This class is designed to be used with the `BasicTimeSeries` class, but (hopefully) is general enough to be used in other situations.

3.11.2 Constructor

To construct a `Day` instance:

```
public Day(int day, int month, int year);  
Creates a new Day instance. The year argument should be in the range  
1900 to 9999.
```

To create a `Day` instance based on a `SerialDate`:

```
public Day(SerialDate day);  
Creates a new Day instance.
```

To create a `Day` instance based on a `Date`:

```
public Day(Date time);  
Creates a new Day instance.
```

The default constructor creates a `Day` instance based on the current system date:

```
public Day();  
Creates a new Day instance for the current system date.
```

3.11.3 Methods

To access the day:

```
public SerialDate getDay();  
Returns the day as a SerialDate.
```

There is no method to *set* the day, because this class is immutable.

Given a `Day` object, you can create an instance representing the previous day or the next day:

```
public TimePeriod previous();  
Returns the previous day, or null if the lower limit of the range is reached.  
  
public TimePeriod next();  
Returns the next day, or null if the upper limit of the range is reached.
```

To convert a `Day` object to a `String` object:

```
public String toString();  
Returns a string representing the day.  
  
public static Day parseDay(String s) throws TimePeriodFormatException;  
Parses the string and, if possible, returns a Day object.
```

3.11.4 Notes

In the current implementation, the day can be in the range 1-Jan-1900 to 31-Dec-9999.

The `Day` class is immutable. This is a requirement for all `TimePeriod` subclasses.

See Also:

`TimePeriod`, `BasicTimeSeries`, `SerialDate`.

3.12 DefaultCategoryDataset

A quick and dirty implementation of the `CategoryDataset` interface. This class is due for a rewrite.

See Also

`CategoryDataset`

3.13 DefaultPieDataset

3.13.1 Overview

A convenient implementation of the `PieDataset` interface.

3.13.2 Constructors

The default constructor creates an empty pie dataset:

```
public DefaultPieDataset();  
Creates a new dataset, initially empty.  
  
public DefaultPieDataset(Collection values);  
Creates a new dataset containing the values supplied. Section names are  
automatically generated.
```

3.13.3 Methods

To get the value for a particular category:

```
public Number getValue(Object category);  
Returns the number associated with a category. This method can return  
null.
```

To set the value for a particular category:

```
public void setValue(Object category, Number value);  
Sets the number associated with a category.
```

3.13.4 Notes

The dataset can contain `null` values.

See Also

`PieDataset`.

3.14 DefaultXYDataset

A quick and dirty implementation of the `XYDataset` interface. This class is in the process of being replaced by `XYSeriesCollection`.

See Also

`XYDataset`

3.15 DomainInfo

3.15.1 Overview

An interface that provides information about the minimum and maximum values in a dataset's domain.

3.15.2 Methods

To get the minimum value in the dataset's domain:

```
public Number getMinimumDomainValue();  
Returns the minimum value in the dataset's domain.
```

To get the maximum value in the dataset's domain:

```
public Number getMaximumDomainValue();  
Returns the maximum value in the dataset's domain.
```

3.15.3 Notes

It is not mandatory for a dataset to implement this interface. However, sometimes it is necessary to calculate the minimum and maximum values in a dataset. Without knowing the internal structure of a dataset, the only means of determining this information is iteration over the entire dataset. If there is a more efficient way to determine the values for your data structures, then you can implement this interface and provide the values directly.

See Also

`RangeInfo`.

3.16 HighLowDataset

An extension of the `XYDataset` interface, that supplies data in the form of "high/low, open/close" items.

```
public Number getHighValue(int series, int item);  
Returns the high value for an item within a series.  
public Number getLowValue(int series, int item);  
Returns the low value for an item within a series.  
public Number getOpenValue(int series, int item);  
Returns the open value for an item within a series.  
public Number getCloseValue(int series, int item);  
Returns the close value for an item within a series.
```

This interface is used in the JFreeChart library.

3.17 Hour

3.17.1 Overview

A subclass of `TimePeriod` that represents one hour in a particular day. This class is designed to be used with the `BasicTimeSeries` class, but (hopefully) is general enough to be used in other situations.

3.17.2 Constructor

To construct an `Hour` instance:

```
public Hour(int hour, Day day);  
Creates a new Hour instance. The hour argument should be in the range  
1 to 24.
```

To construct an `Hour` instance based on a `java.util.Date`:

```
public Hour(Date time);  
Creates a new Hour instance.
```

A default constructor is provided:

```
public Hour();  
Creates a new Hour instance based on the current system time.
```

3.17.3 Methods

To access the hour:

```
public int getHour();  
Returns the hour (in the range 1 to 24).
```

To access the day:

```
public Day getDay();  
Returns the day.
```

There is no method to *set* the hour or the day, because this class is immutable.

Given a `Hour` object, you can create an instance representing the previous hour or the next hour:

```
public TimePeriod previous();  
Returns the previous hour, or null if the lower limit of the range is reached.  
  
public TimePeriod next();  
Returns the next hour, or null if the upper limit of the range is reached.
```

3.17.4 Notes

The `Hour` class is immutable. This is a requirement for all `TimePeriod` subclasses.

See Also:

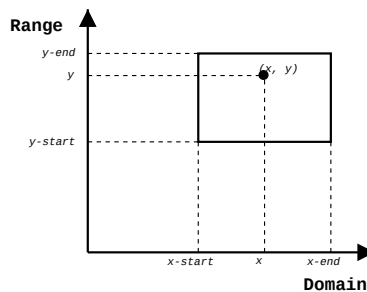
`TimePeriod`, `BasicTimeSeries`, `Day`.

3.18 IntervalXYDataset

An extension of the `XYDataset` interface. Additional methods are provided to define an interval around the X and Y values:

```
public Number getStartXValue(int series, int item);  
Returns the starting x-value for an item within a series.  
public Number getEndXValue(int series, int item);  
Returns the ending x-value for an item within a series.  
public Number getStartYValue(int series, int item);  
Returns the starting y-value for an item within a series.  
public Number getEndYValue(int series, int item);  
Returns the ending y-value for an item within a series.
```

The following diagram illustrates the data dimensions described by the above methods:



3.19 IntervalXYZDataset

A natural extension of the `IntervalXYDataset` interface.

3.20 Millisecond

3.20.1 Overview

A subclass of `TimePeriod` that represents one millisecond. This class is designed to be used with the `BasicTimeSeries` class, but (hopefully) is general enough to be used in other situations.

3.20.2 Constructors

To construct a `Millisecond` instance:

```
public Millisecond(long millisecond);  
Creates a new Millisecond instance. The millisecond argument uses  
the same encoding as java.util.Date.
```

You can construct a `Millisecond` instance based on a `java.util.Date` instance:

```
public Millisecond(Date time);  
Creates a new Millisecond instance representing the same millisecond as  
the time argument.
```

A default constructor is provided, which creates a `Millisecond` instance based on the current system time:

```
public Millisecond();  
Creates a new Millisecond instance based on the current system time.
```

3.20.3 Methods

To access the millisecond:

```
public int getMillisecond();  
Returns the millisecond.
```

There is no method to *set* the millisecond, because this class is immutable.

Given a `Millisecond` object, you can create an instance representing the previous millisecond or the next millisecond:

```
public TimePeriod previous();  
Returns the previous millisecond, or null if the lower limit of the range is reached.  
  
public TimePeriod next();  
Returns the next millisecond, or null if the upper limit of the range is reached.
```

3.20.4 Notes

The `Millisecond` class is immutable. This is a requirement for all `TimePeriod` subclasses.

See Also:

`TimePeriod`, `BasicTimeSeries`, `java.util.Date`.

3.21 Minute

3.21.1 Overview

A subclass of `TimePeriod` that represents one minute in a particular day. This class is designed to be used with the `BasicTimeSeries` class, but (hopefully) is general enough to be used in other situations.

3.21.2 Constructors

To construct a `Minute` instance:

```
public Minute(int minute, Day day);  
Creates a new Minute instance. The minute argument should be in the range 1 to 24*60.
```

To construct a `Minute` instance based on a `java.util.Date`:

```
public Minute(Date time);  
Creates a new Minute instance.
```

A default constructor is provided:

```
public Minute();  
Creates a new Minute instance, based on the current system time.
```

3.21.3 Methods

To access the minute:

```
public int getMinute();  
Returns the minute (in the range 1 to 24*60).
```

To access the day:

```
public Day getDay();  
Returns the day.
```

There is no method to *set* the minute or the day, because this class is immutable.

Given a **Minute** object, you can create an instance representing the previous minute or the next minute:

```
public TimePeriod previous();  
Returns the previous minute, or null if the lower limit of the range is  
reached.  
  
public TimePeriod next();  
Returns the next minute, or null if the upper limit of the range is reached.
```

3.21.4 Notes

The **Minute** class is immutable. This is a requirement for all **TimePeriod** subclasses.

See Also:

TimePeriod, **BasicTimeSeries**, **Day**.

3.22 Month

3.22.1 Overview

A subclass of **TimePeriod** that represents one month in a particular year. This class is designed to be used with the **BasicTimeSeries** class, but (hopefully) is general enough to be used in other situations.

3.22.2 Constructors

To construct a **Month** instance:

```
public Month(int month, Year year);  
Creates a new Month instance. The month argument should be in the  
range 1 to 12.  
  
public Month(int month, int year);  
Creates a new Month instance.
```

To construct a `Month` instance based on a `java.util.Date`:

```
public Month(Date time);  
Creates a new Month instance.
```

A default constructor is provided:

```
public Month();  
Creates a new Month instance, based on the current system time.
```

3.22.3 Methods

To access the month:

```
public int getMonth();  
Returns the month (in the range 1 to 12).
```

To access the year:

```
public Year getYear();  
Returns the year.
```

There is no method to *set* the month or the year, because this class is immutable.

Given a `Month` object, you can create an instance representing the previous month or the next month:

```
public TimePeriod previous();  
Returns the previous month, or null if the lower limit of the range is  
reached.  
  
public TimePeriod next();  
Returns the next month, or null if the upper limit of the range is reached.
```

To convert a `Month` object to a `String` object:

```
public String toString();  
Returns a string representing the month.
```

3.22.4 Notes

In the current implementation, the year can be in the range 1900 to 9999.

The `Month` class is immutable. This is a requirement for all `TimePeriod` subclasses.

See Also:

`TimePeriod`, `BasicTimeSeries`, `Year`.

3.23 PieDataset

3.23.1 Overview

The interface for a dataset that associates values with categories.

3.23.2 Methods

Three methods are defined in the interface:

```
public int getCategoryCount();  
Returns the number of categories in the dataset.  
public Set getCategories();  
Returns the set of categories.  
public Number getValue(Object category);  
Returns the value associated with a particular category.
```

3.23.3 Notes

The name of the interface is derived from a common usage for this type of dataset—the creation of pie charts.

There are some convenient methods for creating a `PieDataset` object by slicing a `CategoryDataset`. Refer to the `Datasets` class for more details.

See Also

`DefaultPieDataset`.

3.24 Quarter

3.24.1 Overview

A subclass of `TimePeriod` that represents one quarter in a particular year. This class is designed to be used with the `BasicTimeSeries` class, but (hopefully) is general enough to be used in other situations.

3.24.2 Constructor

To construct a `Quarter` instance:

```
public Quarter(int quarter, Year year);  
Creates a new Quarter instance. The quarter argument should be in the  
range 1 to 4.  
public Quarter(int quarter, int year);  
Creates a new Quarter instance.
```

To construct a `Quarter` instance based on a `java.util.Date`:

```
public Quarter(Date time);  
Creates a new Quarter instance.
```

A default constructor is provided:

```
public Quarter();  
Creates a new Quarter instance based on the current system time.
```

3.24.3 Methods

To access the quarter:

```
public int getQuarter();  
Returns the quarter (in the range 1 to 4).
```

To access the year:

```
public Year getYear();  
Returns the year.
```

There is no method to *set* the quarter or the year, because this class is immutable.

Given a **Quarter** object, you can create an instance representing the previous quarter or the next quarter:

```
public TimePeriod previous();  
Returns the previous quarter, or null if the lower limit of the range is  
reached.  
  
public TimePeriod next();  
Returns the next quarter, or null if the upper limit of the range is reached.
```

To convert a **Quarter** object to a **String** object:

```
public String toString();  
Returns a string representing the quarter.
```

3.24.4 Notes

In the current implementation, the year can be in the range 1900 to 9999.

The **Quarter** class is immutable. This is a requirement for all **TimePeriod** subclasses.

See Also:

TimePeriod, **BasicTimeSeries**, **Year**.

3.25 RangeInfo

3.25.1 Overview

An interface that provides information about the minimum and maximum values in a dataset's range.

3.25.2 Methods

To get the minimum value in the dataset's range:

```
public Number getMinimumRangeValue();  
Returns the minimum value in the dataset's range.
```

To get the maximum value in the dataset's range:

```
public Number getMaximumRangeValue();  
Returns the maximum value in the dataset's range.
```

3.25.3 Notes

It is not mandatory for a dataset to implement this interface. However, sometimes it is necessary to calculate the minimum and maximum values in a dataset. Without knowing the internal structure of a dataset, the only means of determining this information is iteration over the entire dataset. If there is a more efficient way to determine the values for your data structures, then you can implement this interface and provide the values directly.

See Also

`DomainInfo`.

3.26 Second

3.26.1 Overview

A subclass of `TimePeriod` that represents one second in a particular day. This class is designed to be used with the `BasicTimeSeries` class, but (hopefully) is general enough to be used in other situations.

3.26.2 Constructors

To construct a `Second` instance:

```
public Second(int second, Day day);
```

Creates a new `Second` instance. The `second` argument should be in the range 1 to 24*60*60.

To construct a `Second` instance based on a `java.util.Date`:

```
public Second(Date date);
```

Creates a new `Second` instance.

A default constructor is provided:

```
public Second();
```

Creates a new `Second` instance based on the current system time.

3.26.3 Methods

To access the second:

```
public int getSecond();
```

Returns the second (in the range 1 to 24*60*60).

To access the day:

```
public Day getDay();
```

Returns the day.

There is no method to *set* the second or the day, because this class is immutable. Given a `Second` object, you can create an instance representing the previous second or the next second:

```
public TimePeriod previous();  
Returns the previous second, or null if the lower limit of the range is  
reached.  
  
public TimePeriod next();  
Returns the next second, or null if the upper limit of the range is reached.
```

3.26.4 Notes

The `Second` class is immutable. This is a requirement for all `TimePeriod` subclasses.

See Also:

`TimePeriod`, `BasicTimeSeries`, `Day`.

3.27 SeriesChangeListener

The interface through which series change notifications are posted.

3.28 SeriesDataset

3.28.1 Overview

A base interface that defines a dataset containing zero, one or many data series.

3.28.2 Methods

The methods in the interface are:

```
public int getSeriesCount();  
Returns the number of series in the dataset.  
public String getSeriesName(int series);  
Returns the name of the series with the specified index (zero based).
```

3.28.3 Notes

This interface is extended by `CategoryDataset` and `XYDataset`.

See Also:

`CategoryDataset`, `XYDataset`.

3.29 SeriesException

An exception generated by a series. For example, a time series will not allow duplicate time periods—attempting to add a duplicate time period will throw a `SeriesException`.

3.30 Statistics

3.30.1 Overview

Provides some static utility methods for calculating statistics.

3.30.2 Methods

To calculate the average of an array of `Number` objects:

```
public static double getAverage(Number[] data);
```

Returns the average of an array of numbers.

To calculate the standard deviation of an array of `Number` objects:

```
public static double getStdDev(Number[] data);
```

Returns the standard deviation of an array of numbers.

To calculate a least squares regression line through an array of data:

```
public static double[] getLinearFit(Number[] x_data, Number[] y_data);
```

Returns the intercept (`double[0]`) and slope (`double[1]`) of the linear regression line.

To calculate the slope of a least squares regression line:

```
public static double getSlope(Number[] x_data, Number[] y_data);
```

Returns the slope of the linear regression line.

To calculate the slope of a least squares regression line:

```
public static double getCorrelation(Number[] data1, Number[] data2);
```

Returns the correlation between two sets of numbers.

3.30.3 Notes

This class was written by Matthew Wright.

3.31 SubseriesDataset

A specialised dataset implementation written by Bill Kelemen. To be documented.

3.32 TimePeriod

3.32.1 Overview

An abstract class that represents a fixed period of time. This class and its subclasses are designed to be used with the `BasicTimeSeries` class.

3.32.2 Methods

Given a `TimePeriod` instance, you can create another instance representing the previous time period, or the next time period:

```
public abstract TimePeriod previous();
```

Returns the previous time period, or `null`.

```
public abstract TimePeriod next();
```

Returns the next time period, or `null`.

To get the millisecond at the start, middle and end of the time period (using the same encoding convention as `java.util.Date`):

```
public long getStart();
The first millisecond of the time period.

public long getMiddle();
The middle millisecond of the time period.

public long getEnd();
The last millisecond of the time period.
```

3.32.3 Notes

All `TimePeriod` subclasses are required to be immutable.

Known subclasses include: `Year`, `Quarter`, `Month`, `Week`, `Day`, `Hour`, `Minute`, `Second` and `Millisecond`.

See Also:

`BasicTimeSeries`.

3.33 TimePeriodFormatException

An exception that can be thrown by the methods used to convert time periods to strings, and vice versa.

See Also

`TimePeriod`

3.34 TimeSeriesCollection

3.34.1 Overview

A collection of `TimeSeries` objects. The collection may contain zero, one or many time series.

`TimeSeriesCollection` extends `AbstractSeriesDataset` to provide some of the basic series information.

The collection implements the `IntervalXYDataset` interface (and, therefore, the `XYDataset` interface) and can be used as a convenient dataset for the JFreeChart library.

3.34.2 Constructors

You can construct a `TimeSeriesCollection` in several different ways:

```
public TimeSeriesCollection();
Creates a new time series collection, initially empty.

public TimeSeriesCollection(BasicTimeSeries series);
Creates a new time series collection, containing a single time series.
```

Once a collection has been constructed, you are free to add additional time series to the collection. There are not yet any methods for removing a series from a collection (possibly to be implemented in the future).

3.34.3 Methods

To find out how many time series objects are in the collection:

```
public int getSeriesCount();
```

Returns the number of time series objects in the collection.

To get a reference to a particular series:

```
public BasicTimeSeries getSeries(int series);
```

Returns a reference to a series in the collection.

To get the name of a series:

```
public String getSeriesName(int series);
```

Returns the name of a series in the collection. This method is provided for convenience.

To add a series to the collection:

```
public void addSeries(BasicTimeSeries series);
```

Adds the series to the collection. Registered listeners are notified that the collection has changed.

To get the number of items in a series:

```
public int getItemCount(int series);
```

Returns the number of items in a series. This method is part of the `XYDataset` interface.

3.34.4 Notes

This class implements the `XYDataset` and `IntervalXYDataset` interfaces.

See Also:

`AbstractSeriesDataset`, `BasicTimeSeries`, `XYDataset` and `IntervalXYDataset`.

3.35 TimeSeriesDataPair

Associates a numerical value with a time period. This class is used by the `TimeSeries` class.

There are a number of important features. First, the class implements the `Comparable` interface, allowing data items to be sorted into time order using standard Java API calls. Second, the instances of this class can be easily cloned. Third, the time period element is immutable, so that when a collection of objects is held in sorted order, the sorted property cannot inadvertently be broken.

See Also

`TimeSeries`

3.36 TimeSeriesTableModel

An initial attempt to display a time series in a `JTable`.

3.37 Values

An interface for accessing a set of values. This hasn't been used for anything yet...but the idea was to create a simple data structure that could be passed to a variety of statistical methods (for example, a method that calculates frequency distributions, returning an appropriate dataset for constructing a histogram). More work to be done...

3.38 Week

3.38.1 Overview

A subclass of `TimePeriod` that represents one week in a particular year. This class is designed to be used with the `BasicTimeSeries` class, but (hopefully) is general enough to be used in other situations.

3.38.2 Constructors

To construct a `Week` instance:

```
public Week(int week, Year year);  
Creates a new Week instance. The week argument should be in the range  
1 to 52.  
  
public Week(int week, int year);  
Creates a new Week instance.
```

To construct a `Week` instance based on a `java.util.Date`:

```
public Week(Date time);  
Creates a new Week instance.
```

A default constructor is provided:

```
public Week();  
Creates a new Week instance based on the current system time.
```

3.38.3 Methods

To access the week:

```
public int getWeek();  
Returns the week (in the range 1 to 52).
```

To access the year:

```
public Year getYear();  
Returns the year.
```

There is no method to *set* the week or the year, because this class is immutable. Given a `Week` object, you can create an instance representing the previous week or the next week:

```
public TimePeriod previous();
Returns the previous week, or null if the lower limit of the range is reached.

public TimePeriod next();
Returns the next week, or null if the upper limit of the range is reached.
```

To convert a `Week` object to a `String` object:

```
public String toString();
Returns a string representing the week.
```

3.38.4 Notes

In the current implementation, the year can be in the range 1900 to 9999.

The `Week` class is immutable. This is a requirement for all `TimePeriod` subclasses.

See Also:

`TimePeriod`, `BasicTimeSeries`, `Year`.

3.39 XYDatapair

Associates a numerical value with another numerical value. This class is analagous to the `TimeSeriesDataPair` class.

3.40 XYDataset

3.40.1 Overview

An interface that defines a collection of data in the form of (x, y) values. The dataset can consist of zero, one or many data series. Each series can have (x, y) values that are completely independent of the other series in the dataset.

3.40.2 Methods

The methods in the interface are:

```
public int getItemCount(int series);
Returns the number of data items in a series.

public Number getXValue(int series, int item);
Returns an x-value for a series.

public Number getYValue(int series, int item);
Returns a y-value for a series (possibly null).
```

3.40.3 Notes

`JFreeChart` uses this interface to obtain data for drawing charts.

See Also:

`SeriesDataset`, `DefaultXYDataset`, `IntervalXYDataset`.

3.41 XYSeries

A series of (x, y) data items. Analogous to the `TimeSeries` class.

3.42 XYSeriesCollection

A collection of `XYSeries` objects. This class implements the `XYDataset` interface, so can be used very conveniently with `JFreeChart`.

3.43 XYZDataset

A natural extension of the `XYDataset` interface.

3.44 Year

3.44.1 Overview

A subclass of `TimePeriod` that represents one year. This class is designed to be used with the `TimeSeries` class, but is (hopefully) general enough to be used in other situations.

3.44.2 Constructors

To construct a `Year` instance:

```
public Year(int year);  
Creates a new Year instance. The year argument should be in the range  
1900 to 9999.
```

To construct a `Year` instance based on a `java.util.Date`:

```
public Year(Date time);  
Creates a new Year instance.
```

A default constructor is provided:

```
public Year();  
Creates a new Year instance based on the current system time.
```

3.44.3 Methods

To access the year:

```
public int getYear();  
Returns the year.
```

There is no method to *set* the year, because this class is immutable.

Given a `Year` object, you can create an instance representing the previous year or the next year:

```
public TimePeriod previous();  
Returns the previous year, or null if the lower limit of the range is reached.  
  
public TimePeriod next();  
Returns the next year, or null if the upper limit of the range is reached.
```

To convert a **Year** object to a **String** object, or vice versa:

```
public String toString();  
Returns a string representing the year.  
  
public static Year parseYear(String s) throws TimePeriodFormatException;  
Parses the string and, if possible, returns a Year object.
```

3.44.4 Notes

In the current implementation, the year can be in the range 1900 to 9999.

The **Year** class is immutable. This is a requirement for all **TimePeriod** subclasses.

See Also:

TimePeriod, **TimeSeries**.

4 Package: com.jrefinery.date

This package contains classes that relate to processing dates.

4.1 AnnualDateRule

An *annual date rule* is a function that returns a date when given a year as an argument. This abstract base class defines the standard behaviour for all annual date rules.

```
public SerialDate getDate(int yyyy);  
Returns the date for the specified year. Note that it is acceptable for this  
method to return null.
```

There are several known subclasses of `AnnualDateRule`:

- `DayAndMonthRule` - generates a date that falls on a specific day and month every year;
- `DayOfWeekInMonthRule` - for example, the third Wednesday in September every year;
- `EasterSundayRule` - an algorithm for calculating the date of Easter Sunday for a given year;
- `RelativeDayOfWeekRule` - a day-of-the-week relative to a subrule (for example, the Friday preceding Easter Sunday).

See Also

`SerialDate`

4.2 DayAndMonthRule

An annual date rule that returns the same day and month for every year (for example, 25 December).

Constructors The standard constructor is defined as follows:

```
public DayAndMonthRule(int dayOfMonth, int month);  
The dayOfMonth parameter should be in the range 1 to 31 and the month  
parameter in the range 1 to 12 (you can use the constants SerialDate.JANUARY,  
SerialDate.FEBRUARY...).
```

There is also a default constructor which returns a `DayAndMonthRule` for 1 January.

See Also

`AnnualDateRule`

4.3 DayOfWeekInMonthRule

An annual date rule based on when a specific day of the week falls within a month—for example, the “third Wednesday in September”. You can specify any day of the week along with the count specifier `FIRST_WEEK_IN_MONTH`, `SECOND_WEEK_IN_MONTH`, `THIRD_WEEK_IN_MONTH`, `FOURTH_WEEK_IN_MONTH` or `LAST_WEEK_IN_MONTH`.

See Also

`AnnualDateRule`

4.4 `EasterSundayRule`

An annual date rule that returns the date of Easter Sunday for any given year. I obtained the algorithm from the Calendar FAQ at:

<http://www.tondering.dk/claus/calendar.html>.

It is based on an algorithm by Oudin (1940), quoted in “Explanatory Supplement to the Astronomical Almanac”, P. Kenneth Seidelmann, editor.

See Also

`AnnualDateRule`

4.5 `RelativeDayOfWeekRule`

An annual date rule calculated by finding the date that falls on a certain day-of-the-week preceding, following or nearest to the date produced by another annual date rule. For example, this date rule can return “the Monday following Easter Sunday”, or “the Friday nearest to 22 January”.

Constructor The constructor is defined as follows:

```
public DayOfWeekAnnualDateRule(AnnualDateRule reference, int dayOfWeek,
int relative);
```

The `dayOfWeek` parameter can be specified using `SerialDate.MONDAY`, `SerialDate.TUESDAY` and so on. The `relative` parameter is one of: `SerialDate.PRECEDING`, `SerialDate.NEAREST` and `SerialDate.FOLLOWING`.

See Also

`AnnualDateRule`

4.6 `SerialDate`

An abstract class for representing dates. A subclass (`SpreadsheetDate`) has been implemented. I use this class when the precision of `java.util.Date` gets in the way, as it sometimes does.

See Also

`SerialDates`, `SpreadsheetDate`;

4.7 `SerialDates`

A collection of utility methods for dealing with dates. Most of the methods are static. The non-static methods assume that you have created a single instance of `SerialDates`.

4.8 SpreadsheetDate

Represents a date using an integer, in a very similar way to the date encoding used in several spreadsheets including Microsoft Excel and Lotus 123. Implements the **SerialDate** interface. This class supports dates in the range 1 January 1900 to 31 December 9999.

For those interested in alternatives to Java's **Date** and **Calendar** classes, I would recommend taking a look at the **BigDate** class from Canadian Mind Products:

<http://www.mindprod.com>

It costs US\$10, including source code (which you can evaluate prior to purchase).

See Also

SerialDate

5 Package: `com.jrefinery.io`

This package is for classes related to input/output functions.

5.1 Files

This class contains a single static utility method:

```
public static File findFileOnClassPath(String name);  
Locates a file by name at any point on the classpath.
```

6 Package: com.jrefinery.layout

This package contains some custom layout managers.

6.1 CenterLayout

Specialised layout manager that places a single component in the center of its parent.

6.2 FormatLayout

A flexible layout manager that formats components in rows.



6.3 LCBLLayout

An earlier (and much simpler) version of FormatLayout.

7 Package: com.jrefinery.ui

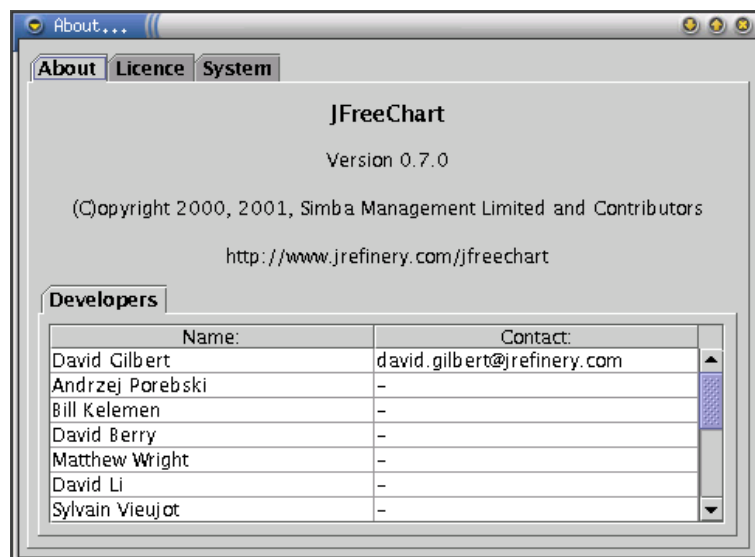
This package contains classes and interfaces that are used to create user interfaces. Please note that some of the classes are “stop-gap” solutions put together for the purpose of creating demonstration applications only—however, it is my intention to replace these, over time, with more robust code.

7.1 AboutFrame

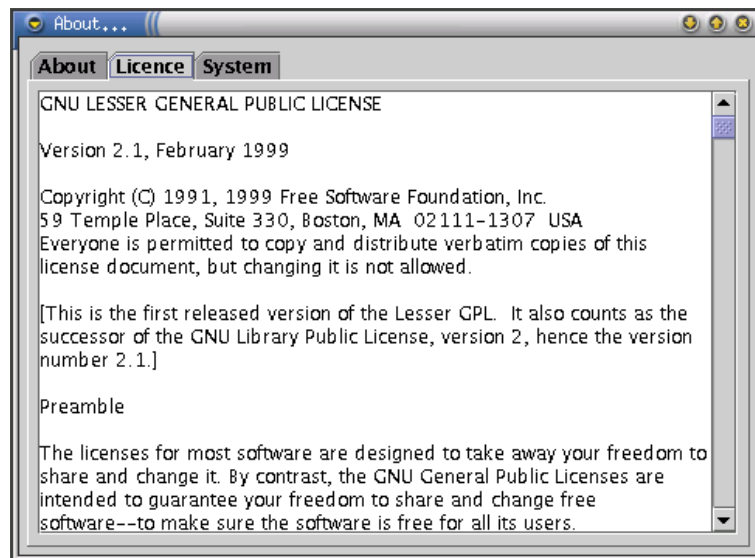
7.1.1 Overview

A utility class for displaying a frame containing information about an application.

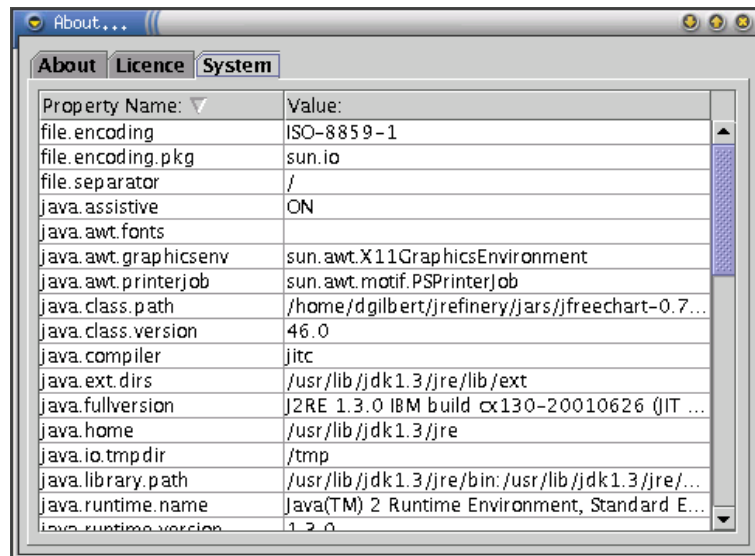
The first tab shows the application name, version, copyright and a list of developers. Here is an example from the JFreeChart demonstration application:



On the second tab in the frame, a licence agreement is displayed. For convenience, the `Licence` class contains the text of the GNU General Public Licence (GPL) and the GNU Lesser General Public Licence (LGPL).



The final tab shows the properties of the current Java Virtual Machine:



This can be useful information for bug reporting.

7.1.2 Constructors

The default constructor:

```
public AboutFrame(String title, String application, String version, String
info, String copyright, String licence, java.util.List contributors);
Creates a new frame for displaying information about an application.
```

See Also

Licences.

7.2 AboutPanel

Not yet documented.

7.3 BevelArrowIcon

An icon that draws an arrow that points up or down. The purpose of this class is to show the sort order of a column in a `JTable`, by drawing the arrow icon next to the column header.

This class is based on code by Nabuo Tamemasa (version 1.0, 26 February 1999) that was posted on <http://www.codeguru.com>. No contact information was provided by the author.

See Also

`SortedTableModel`;

7.4 Contributor

7.4.1 Overview

A simple class representing a contributor to a software project. Used in the `AboutFrame` class.

7.4.2 Constructors

The default constructor:

```
public Contributor(String name, String email);  
Creates a new contributor.
```

7.4.3 Methods

To get the name of the contributor:

```
public String getName();  
Returns the name of the contributor.  
  
public String getEmail();  
To get the e-mail address of the contributor.
```

7.4.4 Notes

The `AboutPanel` class uses a `JTable` to display a list of contributors. The `ContributorsTableModel` class is used to populate the table.

See Also

`ContributorsTableModel`.

7.5 ContributorsPanel

Not yet documented.

7.6 ContributorsTableModel

Not yet documented.

7.7 DateChooserPanel

An incomplete implementation of a control that allows the user to select a date from a calendar-style panel.

7.8 ExtensionFileFilter

A simple file filter that can be added to a `JFileChooser`—the files are filtered on the basis of a single extension.

7.9 FontChooserDialog

A dialog that allows the user to select a font. This class extends `StandardDialog`.

See Also

`FontChooserPanel`;

7.10 FontChooserPanel

A panel containing the user interface for the `FontChooserDialog` class.

See Also

`FontChooserDialog`;

7.11 FontDisplayField

A field for displaying a description of a font. This class extends `JTextField`.

7.12 InsetsChooserPanel

Not yet documented.

7.13 InsetsTextField

Not yet documented.

7.14 IntegerDocument

Not yet documented.

7.15 JRefineryUtilities

A container for a number of useful static methods relating to Swing components.

```
public static void centerFrameOnScreen(JFrame frame);  
Centers the frame on the screen;  
public static void positionFrameOnScreen(JFrame frame, double horizontalPercent,  
double verticalPercent);  
Positions the frame on the screen at a relative position (where 0.5, 0.5 is  
the center of the screen);
```

```
public static void positionFrameRandomly(JFrame frame);
Positions the frame randomly on the screen, but always so that the entire
frame is visible;
public static void centerDialogInParent(JDialog dialog);
Centers the dialog relative to its parent;
public static JPanel createTablePanel(TableModel model);
Returns a JPanel that contains a table;
public static JTable createSystemPropertiesTable();
Returns a JTable containing the current system properties;
```

See Also

SystemPropertiesFrame;

7.16 L1R1ButtonPanel

A panel that contains a single button on the left and a single button on the right. As the panel is resized, the default layout manager takes care of positioning the buttons.

See Also

L1R2ButtonPanel;

7.17 L1R2ButtonPanel

A panel that contains a single button on the left and two buttons on the right. Nested panels and layout managers take care of positioning the buttons as the panel resizes.

See Also

L1R1ButtonPanel;

7.18 L1R3ButtonPanel

A panel that contains a single button on the left and three buttons on the right. Nested panels and layout managers take care of positioning the buttons as the panel resizes.

7.19 Licences

7.19.1 Overview

A utility class that provides access to the full text of the GNU General Public Licence (GPL) and the GNU Lesser General Public Licence. These can be displayed in an `AboutFrame`.

7.19.2 Notes

Other licences may be added in the future.

See Also

AboutFrame.

7.20 NumberCellRenderer

A `TableCellRenderer` that formats numbers in a `JTable` according to the conventions of the default locale.

7.21 PaintSample

A small graphical user interface item that displays a sample of a `Paint` object.

See Also

`StrokeSample`;

7.22 SerialDateChooserPanel

Not yet documented.

7.23 Size2D

Represents the dimensions of an object. Initially I just used `Dimension2D`, but it turns out that there is a bug in the implementation of that class that renders it useless (refer to Bug ID 4189446 on the Java Developer Connection). This class is used as a work-around.

7.24 SortableTable

An extension of `javax.swing.JTable` that uses a `SortableTableModel` and supports sorting on any column (unless prohibited by the table model).

See Also

`SortableTableModel`;

7.25 SortableTableHeaderListener

Handles mouse interaction with the header of a `JTable` that is based on a `SortableTableModel`. The idea is to trigger a sort when the user clicks on a table column header.

This class is adapted from code by Nobuo Tamemasa (version 1.0, 26 February 1999) posted on <http://www.codeguru.com>. No contact information was provided by the author.

See Also

`BevelArrowIcon`, `SortableTableModel`;

7.26 SortableTableModel

An abstract base class for table models that can be sorted on particular columns.

See Also

`SortableTableHeaderListener`;

7.27 SortButtonRenderer

A table cell renderer used for drawing the headers of table columns that are sorted into ascending or descending order.

See Also

`SortableViewModel`;

7.28 StandardDialog

The base class for a standard dialog. Although Swing provides standard dialogs through the `JOptionPane` class, I've decided to implement my own so that I have greater control over the buttons, borders and other user interface items.

7.29 StrokeChooserPanel

7.29.1 Overview

A panel that allows the user to select a particular **Stroke** instance.

7.29.2 Constructors

To create a new instance:

```
public StrokeChooserPanel(StrokeSample current, StrokeSample[] available);  
Creates a new panel for the specified stroke.
```

7.29.3 Notes

This class was hacked together for the demonstration application in JFreeChart. A more robust implementation is required.

See Also

`StrokeSample`.

7.30 StrokeSample

7.30.1 Overview

A panel that displays a sample line drawn using a particular **Stroke**.

7.30.2 Constructors

To create a new instance:

```
public StrokeSample(Stroke stroke);  
Creates a new sample panel for the specified stroke.
```

7.30.3 Notes

This class was hacked together for the demonstration application in JFreeChart. A more robust implementation is required.

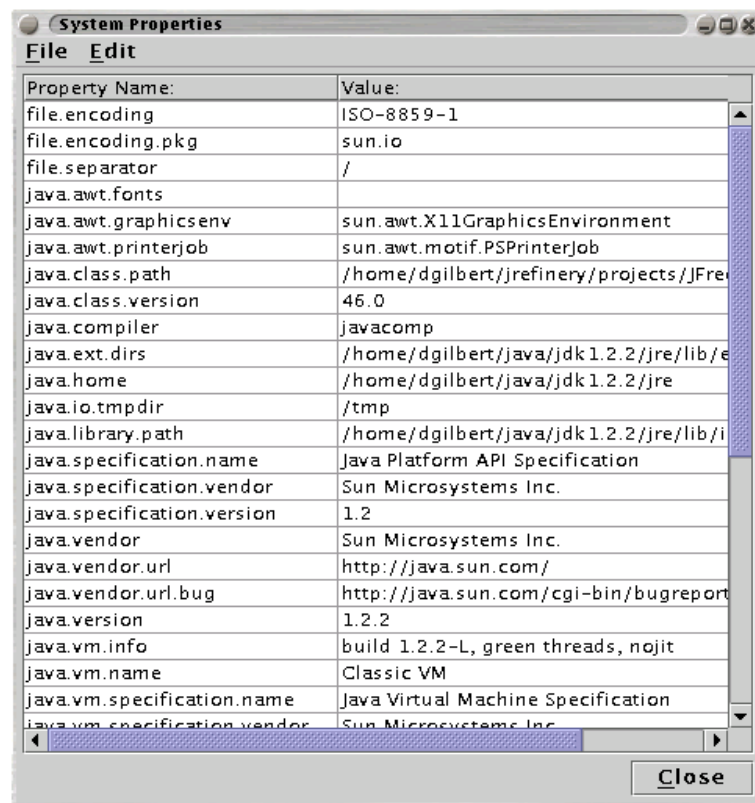
See Also

PaintSample.

7.31 SystemPropertiesFrame

7.31.1 Overview

A frame that is preconfigured to display the properties of the current Java Virtual Machine. For example:



7.31.2 Constructors

The default constructor:

```
public SystemPropertiesFrame(boolean menu);
```

Creates a new frame displaying the system properties. The `menu` argument controls whether or not a menu is provided to allow the user to copy the system properties to the clipboard.

7.31.3 Notes

Now that the `AboutFrame` class includes a tab showing the system properties, this class is less likely to be used.

See Also

SystemPropertiesPanel.

7.32 SystemPropertiesPanel

7.32.1 Overview

A panel containing a `JTable` within a `JScrollPane`, preconfigured to show the properties of the current Java Virtual Machine.

7.32.2 Constructors

The default constructor:

```
public SystemPropertiesPanel();  
Creates a new panel displaying the system properties.
```

7.32.3 Methods

A single method provides the ability to copy the system properties to the clipboard:

```
public void copySystemPropertiesToClipboard();  
Copies the system properties to the clipboard, in tab-delimited format.
```

See Also

`SystemPropertiesTableModel`.

7.33 SystemPropertiesTableModel

7.33.1 Overview

A table model that contains the properties of the current Java Virtual Machine.

7.33.2 Constructors

The default constructor:

```
public SystemPropertiesTableModel();  
Creates a new table model for displaying all the properties of the current  
Java Virtual Machine.
```

7.33.3 Notes

This class extends the `SortableTableModel` class to allow sorting on property names.

In the same source file, two additional classes are defined: `SystemProperty` and `SystemPropertyComparator`.

See Also

`SystemPropertiesPanel`.

7.34 WizardDialog

7.34.1 Overview

A dialog that can be used to present *wizard* sequences.

7.34.2 Notes

This implementation is incomplete and should not be used.

See Also

WizardPanel.

7.35 WizardPanel

7.35.1 Overview

A panel that can be used to present *wizard* sequences.

7.35.2 Notes

This implementation is incomplete and should not be used.

See Also

WizardDialog.

8 Package: com.keypoint

8.1 PNGEncoder

Encodes images in PNG format. This class has been copied into the JCommon class library for convenience only. The original author (J. David Eisenberg) maintains the following web-site for the PNGEncoder:

<http://catcode.com>

A The GNU Lesser General Public Licence

JCommon is licensed under the terms of the GNU Lesser General Public Licence (LGPL). The full text of this licence is reproduced in this appendix. You should read and understand this licence before using JCommon in your own projects.

GNU LESSER GENERAL PUBLIC LICENSE

Version 2.1, February 1999

Copyright (C) 1991, 1999 Free Software Foundation, Inc. 59 Temple Place, Suite 330, Boston, MA 02111-1307 USA Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

[This is the first released version of the Lesser GPL. It also counts as the successor of the GNU Library Public License, version 2, hence the version number 2.1.]

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public Licenses are intended to guarantee your freedom to share and change free software—to make sure the software is free for all its users.

This license, the Lesser General Public License, applies to some specially designated software packages—typically libraries—of the Free Software Foundation and other authors who decide to use it. You can use it too, but we suggest you first think carefully about whether this license or the ordinary General Public License is the better strategy to use in any particular case, based on the explanations below.

When we speak of free software, we are referring to freedom of use, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish); that you receive source code or can get it if you want it; that you can change the software and use pieces of it in new free programs; and that you are informed that you can do these things.

To protect your rights, we need to make restrictions that forbid distributors to deny you these rights or to ask you to surrender these rights. These restrictions translate to certain responsibilities for you if you distribute copies of the library or if you modify it.

For example, if you distribute copies of the library, whether gratis or for a fee, you must give the recipients all the rights that we gave you. You must make sure that they, too, receive or can get the source code. If you link other code with the library, you must provide complete object files to the recipients, so that they can relink them with the library after making changes to the library and recompiling it. And you must show them these terms so they know their rights.

We protect your rights with a two-step method: (1) we copyright the library, and (2) we offer you this license, which gives you legal permission to copy, distribute and/or modify the library.

To protect each distributor, we want to make it very clear that there is no warranty for the free library. Also, if the library is modified by someone else and passed on, the recipients should know that what they have is not the original version, so that the original author's reputation will not be affected by problems that might be introduced by others.

Finally, software patents pose a constant threat to the existence of any free program. We wish to make sure that a company cannot effectively restrict the users of a free program by obtaining a restrictive license from a patent holder. Therefore, we insist that any patent license obtained for a version of the library must be consistent with the full freedom of use specified in this license.

Most GNU software, including some libraries, is covered by the ordinary GNU General Public License. This license, the GNU Lesser General Public License, applies to certain designated libraries, and is quite different from the ordinary General Public License. We use this license for certain libraries in order to permit linking those libraries into non-free programs.

When a program is linked with a library, whether statically or using a shared library, the combination of the two is legally speaking a combined work, a derivative of the original library. The ordinary General Public License therefore permits such linking only if the entire combination fits its criteria of freedom. The Lesser General Public License permits more lax criteria for linking other code with the library.

We call this license the “Lesser” General Public License because it does Less to protect the user’s freedom than the ordinary General Public License. It also provides other free software developers Less of an advantage over competing non-free programs. These disadvantages are the reason we use the ordinary General Public License for many libraries. However, the Lesser license provides advantages in certain special circumstances.

For example, on rare occasions, there may be a special need to encourage the widest possible use of a certain library, so that it becomes a de-facto standard. To achieve this, non-free programs must be allowed to use the library. A more frequent case is that a free library does the same job as widely used non-free libraries. In this case, there is little to gain by limiting the free library to free software only, so we use the Lesser General Public License.

In other cases, permission to use a particular library in non-free programs enables a greater number of people to use a large body of free software. For example, permission to use the GNU C Library in non-free programs enables many more people to use the whole GNU operating system, as well as its variant, the GNU/Linux operating system.

Although the Lesser General Public License is Less protective of the users’ freedom, it does ensure that the user of a program that is linked with the Library has the freedom and the wherewithal to run that program using a modified version of the Library.

The precise terms and conditions for copying, distribution and modification follow. Pay close attention to the difference between a “work based on the library” and a “work that uses the library”. The former contains code derived from the library, whereas the latter must be combined with the library in order to run.

TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License Agreement applies to any software library or other program which contains a notice placed by the copyright holder or other authorized party saying it may be distributed under the terms of this Lesser General Public License (also called “this License”). Each licensee is addressed as “you”.

A “library” means a collection of software functions and/or data prepared so as to be conveniently linked with application programs (which use some of those functions and data) to form executables.

The “Library”, below, refers to any such software library or work which has been distributed under these terms. A “work based on the Library” means either the Library or any derivative work under copyright law: that is to say, a work containing the Library or a portion of it, either verbatim or with modifications and/or translated straightforwardly into another language. (Hereinafter, translation is included without limitation in the term “modification”).

“Source code” for a work means the preferred form of the work for making modifications to it. For a library, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the library.

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running a program using the Library is not restricted, and output from such a program is covered only if its contents constitute a work based on the Library (independent of the use of the Library in a tool for writing it). Whether that is true depends on what the Library does and what the program that uses the Library does.

1. You may copy and distribute verbatim copies of the Library’s complete source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the

absence of any warranty; and distribute a copy of this License along with the Library.

You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Library or any portion of it, thus forming a work based on the Library, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

* a) The modified work must itself be a software library. * b) You must cause the files modified to carry prominent notices stating that you changed the files and the date of any change. * c) You must cause the whole of the work to be licensed at no charge to all third parties under the terms of this License. * d) If a facility in the modified Library refers to a function or a table of data to be supplied by an application program that uses the facility, other than as an argument passed when the facility is invoked, then you must make a good faith effort to ensure that, in the event an application does not supply such function or table, the facility still operates, and performs whatever part of its purpose remains meaningful.

(For example, a function in a library to compute square roots has a purpose that is entirely well-defined independent of the application. Therefore, Subsection 2d requires that any application-supplied function or table used by this function must be optional: if the application does not supply it, the square root function must still compute square roots.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Library, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Library, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Library.

In addition, mere aggregation of another work not based on the Library with the Library (or with a work based on the Library) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may opt to apply the terms of the ordinary GNU General Public License instead of this License to a given copy of the Library. To do this, you must alter all the notices that refer to this License, so that they refer to the ordinary GNU General Public License, version 2, instead of to this License. (If a newer version than version 2 of the ordinary GNU General Public License has appeared, then you can specify that version instead if you wish.) Do not make any other change in these notices.

Once this change is made in a given copy, it is irreversible for that copy, so the ordinary GNU General Public License applies to all subsequent copies and derivative works made from that copy.

This option is useful when you wish to copy part of the code of the Library into a program that is not a library.

4. You may copy and distribute the Library (or a portion or derivative of it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange. If distribution of object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place satisfies the requirement to distribute the source code, even though third parties are not compelled to copy the source along with the object code.

5. A program that contains no derivative of any portion of the Library, but is designed to work with the Library by being compiled or linked with it, is called

a “work that uses the Library”. Such a work, in isolation, is not a derivative work of the Library, and therefore falls outside the scope of this License.

However, linking a “work that uses the Library” with the Library creates an executable that is a derivative of the Library (because it contains portions of the Library), rather than a “work that uses the library”. The executable is therefore covered by this License. Section 6 states terms for distribution of such executables.

When a “work that uses the Library” uses material from a header file that is part of the Library, the object code for the work may be a derivative work of the Library even though the source code is not. Whether this is true is especially significant if the work can be linked without the Library, or if the work is itself a library. The threshold for this to be true is not precisely defined by law. If such an object file uses only numerical parameters, data structure layouts and accessors, and small macros and small inline functions (ten lines or less in length), then the use of the object file is unrestricted, regardless of whether it is legally a derivative work. (Executables containing this object code plus portions of the Library will still fall under Section 6.)

Otherwise, if the work is a derivative of the Library, you may distribute the object code for the work under the terms of Section 6. Any executables containing that work also fall under Section 6, whether or not they are linked directly with the Library itself.

6. As an exception to the Sections above, you may also combine or link a “work that uses the Library” with the Library to produce a work containing portions of the Library, and distribute that work under terms of your choice, provided that the terms permit modification of the work for the customer’s own use and reverse engineering for debugging such modifications.

You must give prominent notice with each copy of the work that the Library is used in it and that the Library and its use are covered by this License. You must supply a copy of this License. If the work during execution displays copyright notices, you must include the copyright notice for the Library among them, as well as a reference directing the user to the copy of this License. Also, you must do one of these things:

- * a) Accompany the work with the complete corresponding machine-readable source code for the Library including whatever changes were used in the work (which must be distributed under Sections 1 and 2 above); and, if the work is an executable linked with the Library, with the complete machine-readable “work that uses the Library”, as object code and/or source code, so that the user can modify the Library and then relink to produce a modified executable containing the modified Library. (It is understood that the user who changes the contents of definitions files in the Library will not necessarily be able to recompile the application to use the modified definitions.)

- * b) Use a suitable shared library mechanism for linking with the Library. A suitable mechanism is one that (1) uses at run time a copy of the library already present on the user’s computer system, rather than copying library functions into the executable, and (2) will operate properly with a modified version of the library, if the user installs one, as long as the modified version is interface-compatible with the version that the work was made with.

- * c) Accompany the work with a written offer, valid for at least three years, to give the same user the materials specified in Subsection 6a, above, for a charge no more than the cost of performing this distribution.

- * d) If distribution of the work is made by offering access to copy from a designated place, offer equivalent access to copy the above specified materials from the same place.

- * e) Verify that the user has already received a copy of these materials or that you have already sent this user a copy.

For an executable, the required form of the “work that uses the Library” must include any data and utility programs needed for reproducing the executable from it. However, as a special exception, the materials to be distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating

system on which the executable runs, unless that component itself accompanies the executable.

It may happen that this requirement contradicts the license restrictions of other proprietary libraries that do not normally accompany the operating system. Such a contradiction means you cannot use both them and the Library together in an executable that you distribute.

7. You may place library facilities that are a work based on the Library side-by-side in a single library together with other library facilities not covered by this License, and distribute such a combined library, provided that the separate distribution of the work based on the Library and of the other library facilities is otherwise permitted, and provided that you do these two things:

- * a) Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities. This must be distributed under the terms of the Sections above.

- * b) Give prominent notice with the combined library of the fact that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.

8. You may not copy, modify, sublicense, link with, or distribute the Library except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, link with, or distribute the Library is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

9. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Library or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Library (or any work based on the Library), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Library or works based on it.

10. Each time you redistribute the Library (or any work based on the Library), the recipient automatically receives a license from the original licensor to copy, distribute, link with or modify the Library subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties with this License.

11. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Library at all. For example, if a patent license would not permit royalty-free redistribution of the Library by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Library.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply, and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

12. If the distribution and/or use of the Library is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Library under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

13. The Free Software Foundation may publish revised and/or new versions of the Lesser General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Library specifies a version number of this License which applies to it and “any later version”, you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Library does not specify a license version number, you may choose any version ever published by the Free Software Foundation.

14. If you wish to incorporate parts of the Library into other free programs whose distribution conditions are incompatible with these, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

15. BECAUSE THE LIBRARY IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE LIBRARY, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE LIBRARY “AS IS” WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE LIBRARY IS WITH YOU. SHOULD THE LIBRARY PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE LIBRARY AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE LIBRARY (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE LIBRARY TO OPERATE WITH ANY OTHER SOFTWARE), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS